

Guide to Upgrade SAP Commerce for Apple Pay URL Validation

Guide to Upgrade SAP Commerce for Apple Pay URL Validation	1
Steps to Upgrade for Apple Pay URL Validation	2
1. Update the ApplePayController.java.....	2

Steps to Upgrade for Apple Pay URL Validation

1. Update the ApplePayController.java

File Path:

hybris/bin/<env>/isvpaymentaddon/acceleratoraddon/web/src/isv/sap/payment/addon/controllers/pages/checkout/payment/alternative/ApplePayController.java

Add below import statement:

```
import java.net.MalformedURLException;  
import java.net.URL;  
import java.util.Arrays;  
import java.util.HashSet;  
import java.util.Set;  
import de.hybris.platform.acceleratorstorefrontcommons.annotations.RequireHardLogin;  
import org.springframework.http.HttpStatus;
```

Add the method:

```
private static final Set<String> ALLOWED_APPLE_PAY_HOSTS = new  
HashSet<>(Arrays.asList(  
    "apple-pay-gateway.apple.com",  
    "cn-apple-pay-gateway.apple.com",  
    "apple-pay-gateway-cert.apple.com",  
    "cn-apple-pay-gateway-cert.apple.com"  
));
```

Replace the method validateMerchant:

```
@RequireHardLogin  
@RequestMapping(value = "/validate", method = GET, produces =  
MediaType.APPLICATION_JSON_VALUE)  
@ResponseBody  
public ResponseEntity<Map<String, Object>> validateMerchant(final String  
validationUrl)  
{  
    // Validate that validationUrl is provided  
    if (validationUrl == null || validationUrl.trim().isEmpty())
```

```

{
    LOG.warn("Apple Pay validation attempted with empty validationUrl");
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
}

// Parse and validate the URL to prevent SSRF attacks
try
{
    final URL url = new URL(validationUrl);

    // Enforce HTTPS protocol
    if (!"https".equalsIgnoreCase(url.getProtocol()))
    {
        LOG.warn("Apple Pay validation attempted with non-HTTPS URL: {}");
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
    }

    // Validate against Apple Pay domain allowlist to prevent SSRF attacks
    final String host = url.getHost().toLowerCase();
    if (!ALLOWED_APPLE_PAY_HOSTS.contains(host))
    {
        LOG.warn("Apple Pay validation attempted with unauthorized host: {}");
        return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    // Additional security check: ensure no credentials in URL
    if (url.getUserInfo() != null)
    {
        LOG.warn("Apple Pay validation attempted with credentials in URL");
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
    }

    return
    ResponseEntity.ok(applePayPaymentFacade.createApplePaySession(validationUrl));
}
catch (final MalformedURLException e)
{
    LOG.warn("Apple Pay validation attempted with malformed URL: {}", e);
}

```

```
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
    }
    catch (final Exception e)
    {
        LOG.error("Error during Apple Pay merchant validation", e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
    }
}
```

NOTE: <env> holds either “b2b” or “b2c”